

版本与分支感知的多状态记忆

方案介绍与测试结论

TencentDB Agent Memory · 比赛提交材料 · 2026 年 9 月 5 日

核心结论。 本方案将“事实是否有效”与“写入时间是否最新”分开建模，在分支、工作树和并行任务中保留各自有效的记忆，并在真实召回链路中选择当前适用的状态。本次优化修复了候选提前截断、异常路径绕过版本选择和 Git 坐标缓存过期等问题。在新增的 180 个候选池内案例中，精确选择由旧版的 45/180 提升到 180/180；新增 12 个池外案例仍未召回，作为明确边界报告。

新一轮 70 题版本能力评测使用优化代码生成的生产上下文，并重新调用 MiniMax-M3 与 deepseek-v4-flash 生成答案、交叉评分。全局混合召回的 criterion accuracy 为 95.00%，优化版为 100.00%，配对差值为 +5.00 个百分点，95% 聚类 bootstrap 区间为 [2.14, 8.21]。这是受控版本场景的结果，不代表任意编程任务的总体正确率。

1. 课题问题与方案定位

长期协作中的记忆会更新，但软件项目通常不存在唯一的“当前版本”。例如 main 已使用 Java 21，release 仍使用 Java 17；一个 worktree 临时关闭缓存，另一个 worktree 保持默认配置；两个 Agent task 分别验证不同假设。这些内容可以互相矛盾，却同时正确。全局最新值会覆盖仍有用的旧分支事实，简单的旧→新纠错链也不能表达并行有效性。

本方案以现有 MemoryCore 的 L0→L1 写入、FTS5/向量/混合检索和提示注入为基础，在记忆元数据中附加作用域坐标。它既不替换 Reader，也不依靠运行时大模型决定哪些分支事实可以覆盖其他事实；作用域匹配、候选排序和有界状态选择均由确定性代码执行。

交付成果

证据层	本次完成内容
实现与回归	独立分支；106 项单元/集成测试，162 项评测框架测试通过
新增压力对照	三组各 204 次真实 SQLite 召回，共 612 次；逐例输出、协议和复算脚本
既有场景重放	110 题、260 次 L1 写入、330 次生产召回；作用域与兼容性检查通过
新模型评测	500 次 Reader、500 次交叉 Judge；独立验证通过

版本标识。 实现提交 caa96f3；评分前冻结提交 689c27a；交付分支 codex/competition-multistate-final。旧版比较固定在 25a025b，不将不同版本或不同实验面板混为同一个结果。

2. 方案原理：同域更新，跨域并存

2.1 有效域模型

每条 L1 记忆携带 schemaVersion、repositoryId、scopeLevel，以及相应的 branch、worktreeId、taskId 和 commitSha。仓库与工作树标识由真实 Git 信息生成不含原始路径的短哈希；commit 用于溯源。分支是可变工作流，detached HEAD 是具体提交快照，后者必须保留完整提交号参与有效性与去重判断。

层级	有效域键	示例
repository	repo	项目通用约定
branch	repo + branch	release 的编译环境
worktree	repo + branch + worktree	当前工作树的临时配置
task	repo + branch + worktree + task	当前并行任务的假设

repository 事实可用于同仓库各分支；branch 事实还需分支相同；worktree、task 事实逐级增加条件。对于 detached HEAD，提交号必须一致。写入域判断采用精确键相等，不能把“在当前任务中可用”误当成“允许在当前任务中覆盖”。

2.2 写入、反馈与召回闭环

捕获。宿主在 recall/capture 入口传入 workspaceDir、稳定 taskId 或显式 versionContext。显式坐标优先，自动探测从真实 Git 读取。session + task 持久化用于后续抽取任务延续，避免同一会话中的并行任务互相覆盖坐标。

写入。抽取出的记忆经 writeMemory 保存，坐标写入 metadata。去重候选只在完全相同的有效域内匹配。对于更新或合并，先读取并验证前驱域，成功写入后继后才发布纠错事件；跨分支目标不会被新域写入删除，也不会被连接为同一纠错链。

召回。原检索器生成有界候选池，生命周期解析只处理有证据的纠错关系，随后执行版本选择，最后限制条数和提示预算。普通问题保留与当前环境匹配的状态；显式比较、迁移、回归或历史问题可返回同仓库的多个带来源标签的状态。

反馈。当前实现的可信输入来自宿主坐标、成功写入与经过验证的前驱 ID。运行日志记录候选、抑制、意图、回退与返回状态。离线评测反馈用于调整和验证策略，不直接把模型 Judge 的意见作为跨域覆盖指令。无证据的自然语言自动构链不属于本方案已验证能力。

2.3 关键不变量

状态身份随写入和召回保留；更新只在同域发生；任何纠错回退之后仍须进行版本筛选；有缺陷的作用域元数据不自动降级为 legacy；最终返回条数不超过 k。作用域限定的是事实有效性，用户和租户访问控制仍由原存储后端负责。

3. 本次策略与代码优化

3.1 让候选过取真正生效

旧流程虽然设置 4 倍过取，却在生命周期解析阶段把候选再次截回 Top-k。设 $k=2$ ，前两条均来自 main，第三条才属于当前 release；第三条会在版本筛选之前消失。此时系统可以避免返回错误分支，但也无法返回已检索到的正确状态。

优化后，解析阶段保留完整的有界候选池 C，版本选择后再取最终 Top-k。简写为“检索 Top-(4k) → 有界纠错 → 版本选择 → Top-k → 提示预算”。处理的候选增加，输出条数上限保持不变。时间复杂度主要为候选筛选和排序 $O(C \log C)$ ，纠错搜索仍受 maxHops、maxExpansions 和 timeoutMs 约束。



3.2 将失败回退与版本约束分离

纠错日志损坏、读取失败或候选来自混合身份域时，可以停止纠错重定向，但不能因此跳过版本选择。优化版统一把回退候选交给选择器；缺少结构化候选时 strict 模式不注入无法验证的文本。没有可用记忆时仍返回决策信息，便于区分“主动弃权”和“未执行召回”。

3.3 修复 Git 坐标与快照身份

原缓存会在两秒内重复使用整个 context，切换分支或提交后可能沿用旧 branch/commit。优化版每次读取 HEAD 和符号引用，缓存只复用仓库、工作树等身份信息；初始化探测前后还会复查快照。返回对象使用副本，防止调用方修改缓存；缓存条目上限为 256。此处增加了 Git 读取成本，换取分支切换后的正确性。

detached HEAD 的写入域加入完整 commitSha。即使宿主把两个快照都表示为 HEAD，或显示前缀相同，不同提交也不会被误判为同一个可覆盖状态。task 层级必须具备 branch、worktree、task 三个维度。

3.4 更精确地理解显式作用域

分支匹配从任意子串改为完整引用边界，避免把 domain 中的 main 当作另一条分支，同时允许分支名后的句号。明确查询 release 历史时只选择该分支。明确询问当前 task/worktree 且有该层级命中时，优先该层级，不把祖先默认值并列成当前答案；普通查询仍可保留不同层级、不同主题的兼容事实。

3.5 保留可审计的兼容路径

新增回归覆盖上述失败触发条件，并检查 strict 关闭时原始前缀不变。补充依赖锁文件、顺序测试入口、实际旧提交对照、逐例指标复算和付费评测预算记录。实现没有根据模型评分反复调参；最终模型评分前已冻结代码对应上下文和协议。

4. 测试设计与指标口径

4.1 新增候选排名压力测试

测试使用真实 SQLite FTS5 和生产 writeMemory、performAutoRecall。对 branch、worktree、task 三个层级，分别设置 $k \in \{1, 2, 4, 8\}$ ；每组写入 $4k+1$ 条并列有效的状态，观察真实 FTS 排序，再逐个改变当前执行坐标，让正确状态遍历每个排名。每组另加一次缺失上下文查询。

因此每个方案包含 180 个候选池内案例、12 个池外案例、12 个缺失坐标案例，共 204 次。三组为：固定旧提交的全局召回、同一旧提交的 strict 版本感知、优化版 strict。旧实现通过动态加载真实旧代码运行，未在评测脚本中重新编写替代算法。三组使用相同数据、k、期望状态和存储路径类型。

指标	定义
精确选择率	返回的记忆 ID 集合与期望集合完全相同的案例比例
期望状态召回率	返回集合覆盖期望集合的比例；缺失坐标案例记为不适用
跨状态污染率	至少返回一条不属于期望集合的记忆的案例比例
注入 token	prependContext 与 appendSystemContext 的 cl100k_base 计数
本地 P50/P95	调用生产召回的墙钟时间，预热一次；token 计数耗时不计入

该面板是针对候选截断缺陷设计的诊断集，排名均匀遍历不代表自然流量分布。池外案例单列，不能用弃权代替正确召回；它们也计入总体结果。

4.2 既有 110 题生产重放与新模型评测

能力面板为 10 个编程场景 $\times$ 7 类查询，共 70 题，覆盖分支、detached worktree、并行 task、比较、迁移、回归和缺失坐标。安全面板来自固定版本 Memora，每个 persona 按固定顺序取 4 题，共 40 题。它是已使用过的兼容性面板，不是新的外部留出集。

生产重放新建真实 Git 仓库及 worktree、50 个 SQLite 数据库，执行 260 次写入、330 次召回。答案层对照沿用全局混合、线性双态、优化版多状态三组；相同案例中完全相同的提示仅生成一次答案并复用，250 个独立提示分别交给两位 Reader，共 500 个答案。

MiniMax-M3 的答案交给 deepseek-v4-flash 判断，反向亦然；禁止同模型自评。criterion accuracy 为每个答案中满足的判据比例；MPA、FAA 分别统计记忆存在和过期内容缺席判据。FAMA= $\max(0, \text{MPA} - \lambda \times (1 - \text{FAA}))$ ， λ 为该题缺席判据占比。先求两位 Reader 的均值，再对案例等权平均。按 10 个场景/persona 进行 5,000 次配对聚类 bootstrap。独立验证器从原始 verdict 重算结果与门槛。最终提交代码再次重放后，优化组全部 110 个输入逐字一致；对照组有 110 个输入仅活动时间变化，差异明细随材料保存。

资料依据：[TencentDB Agent Memory 代码](#)；[Memora 固定修订](#)；本分支冻结协议、逐例 JSON 和独立复算结果。

5. 新增压力测试与工程验证结果

5.1 主要结果

精确集合结果	全局召回	旧版 strict	优化版 strict
候选池内：精确选中	3/180	45/180	180/180
其中 Top-k 之外	0/135	0/135	135/135
候选池之外	0/12	0/12	0/12
缺失坐标：正确弃权	0/12	12/12	12/12
全部案例：集合完全正确	3/204	57/204	192/204

候选池内精确选择从 25.00% 提升到 100.00%，新增找回 135 个正确状态，均来自原 Top-k 之外。优化版 204 个案例中没有跨状态污染，12 个缺失坐标案例均弃权；但池外 12 个案例都漏召回。包含池外案例的总体精确选择率为 94.12%，不是 100%。

5.2 token 与延迟的真实代价

候选池内 180 例	全局召回	旧版 strict	优化版 strict
平均注入 token	560.00	105.33	421.33
平均条目	5.67	0.25	1.00
本地 P50 / ms	0.203	0.269	0.294
本地 P95 / ms	0.342	0.438	0.426

相对全局召回，优化版平均 token 减少 24.76%，条目由 5.67 条减少到 1 条。旧版 strict 的 token 更低，是因为 75% 的池内案例没有返回答案状态；不能把漏召回当成效率收益。优化版处理更多有效候选；本轮延迟高于全局召回，单轮测量的小幅差异不作性能定论。这些数值来自单机、显式坐标、无向量模型调用的热启动测试，不包含 Git 自动探测和在线数据库往返。

5.3 回归与可执行性

新边界测试集在旧提交上共 15 项，13 项失败、2 项通过；优化后全部通过。失败触发覆盖 Top-k 外状态、纠错日志损坏、混合身份域回退、HEAD 缓存、detached 去重、task 坐标缺失、分支子串和历史范围。

完整本地测试 106/106 通过，生命周期评测框架 162/162 通过；修改模块及新增压力脚本的 TypeScript 检查、插件打包通过，另有 4 项无付费调用的预算恢复检查通过。独立复算同时检查 204 例唯一性、三组期望集合一致、条数上限、逐例指标和上下文完整性。这里的“全部测试”指本分支可执行的本地测试范围，不指线上云服务或外部 CI。

6. 新模型复测与历史证据核验

6.1 优化代码对应的 70 题新结果

版本能力面板	全局混合	线性双态	优化版多状态
criterion accuracy	95.00%	70.36%	100.00%
MPA	95.00%	50.71%	100.00%
FAA	95.00%	90.00%	100.00%
FAMA	94.64%	50.36%	100.00%
状态集合精确选择	0.00%	0.00%	100.00%
跨状态污染	100.00%	100.00%	0.00%

优化版相对全局混合的 criterion accuracy 差值为 +5.00 个百分点，95% 区间 [2.14, 8.21]；逐案例统计为 6 改善、64 持平、0 受损。预设的全部效果与安全门槛通过。

线性双态对照把并行状态人为组织为旧→新关系，用于检验这一建模假设；它是机制对照，不是当前领域最强系统。主比较仍以全局混合为参照，新增代码优化收益则由上一页“旧版 strict → 优化版 strict”压力对照证明。

6.2 兼容性与评分完整性

40 题 Memora 面板三组最终提示逐字一致，按协议在同一案例内复用相同提示的答案，FAMA 均为 2.22%。分数很低，说明这批已截取记忆对公开问题的回答支持有限；该面板仅能证明兼容路径不变，不能证明通用记忆质量优秀。

本次模型评测完成 500 次 Reader 与 500 次交叉 Judge，Reader 重试 0 次，Judge 重试 0 次，模型错配 0 次，自评 0 次，unclear verdict 0 条。独立验证的 mismatchCount 为 0。模型响应与逐条判分均随分支提交。

本次返回的累计输入/输出用量为 582,855 tokens。预算工具按每百万输入或输出 token 统一 100 元、忽略缓存折扣作保守记账，合计 58.29 元，低于授权上限 200 元；该数字是预算控制口径，不是供应商实际账单。

6.3 与原报告的关系

原报告记录的 94.29%→100.00% 和 500 条历史判分已独立复算，mismatchCount=0。本报告第 6.1 节使用本次新调用结果，不把旧结果直接当成新实现的答案得分。生产重放与新评测均固定 Memora 数据修订和上下文哈希，历史文件保持独立。

这两个层次回答不同问题：压力测试定位并验证“正确状态已检索到却被截断”的实现缺陷；70 题模型评测检验所选上下文能否支持正确答案。二者不能合并样本量后宣称更高置信度。

7. 部署、交付与结论

7.1 接入方式

在现有 recall 配置中同时开启 lifecycle.enabled 与 versionAwareMode=strict，按宿主需要开启 autoDetectGit。运行时向 recall/capture 传入 workspaceDir 和稳定 taskId；显式 versionContext 可用于已知版本坐标的接入。task 必须包含完整分支与工作树维度。建议从既有参数开始：候选倍率 4、maxVersionStates=6、maxHops=1、maxExpansions=64、minConfidence=0.85、timeoutMs=10。

上述 minConfidence 是可信事件的策略权重阈值，不是经过校准的正确概率。生产默认纠错预算为 10 ms；用于可重复评测的 110 题重放和压力测试固定为 100 ms。两者不能混写成同一运行配置。在线使用时应根据真实后端延迟重新测量。

7.2 交付索引

材料	入口
实现	MemoryCore/src/core/lifecycle/ 与 hooks/auto-recall.ts
新增测试及协议	MemoryCore/benchmarks/competition/；两个新增边界测试文件
一键本地验证	node submission/run-local.mjs
旧版失败复现	node submission/check-regressions.mjs
独立指标复算	python3 submission/validate.py
完整复现说明	submission/README.md
原始证据	submission/evidence/；原有 lifecycle-memory/results/
报告源稿与生成器	submission/report.md；submission/render-report.py

代码分支：[NianJiuZst/TencentDB-Agent-Memory · competition-multistate-final](#)。报告 PDF 位于 [MemoryCore/output/pdf/competition-memory-solution-cn.pdf](#)。

7.3 已验证结论与适用边界

方案已经通过真实 Git/worktree、L1 写入、SQLite/FTS5 召回、版本选择、提示预算和交叉模型评分形成可复现证据链。本次改进修复了旧版严格筛选“安全但漏召回”的关键缺陷，使候选池内正确状态不再因为提前 Top-k 截断而丢失，同时保留并行域隔离与最终输出上限。

现有证据仍有边界：4k 候选池之外的事实可能漏召回；依赖宿主或抽取器提供正确的事实作用域；同一分支不同时间点的语义失效需要可靠纠错证据；尚未验证复杂 merge/cherry-pick 祖先推断、自然流量冲突发生率、在线 TCVD/DB/COS 延迟和完整多轮代码任务的最终成功率。规则意图识别也不等同于通用自然语言理解。上述内容是后续可独立验证的扩展方向。

最终结论。本次提交完成了可运行的多状态记忆实现、针对实际缺陷的策略优化、可复现的实现与答案层测试，以及完整证据和报告。适用场景是同仓库多分支、多工作树和并行 Agent 的长期协作；贡献在于让“记忆适用于哪里”成为可验证的运行条件。